



Introduction to Parallel Programming with CUDA
Johns Hopkins University

Course Material

- Module 1
- Module 2
- Module 3
- Module 4
- Module 5

Grades

Notes

Discussion Forums

Messages

Course Info

Module 5

Lessons learned and tips for assignment

pb peter brooker Learner

6 months ago

Here are some tips:

- make a copy of the starting *.cu. You might need to start over.
- They say only 2 hrs. I think this is for somebody who already knows GPU and C++ inside and out. For me, I burned 5 days. Do not be discouraged. It does force you to pay attention to the details of memory which is worth the investment.
- In executeKernel, the calls do not have any arguments. Look to the kernel definitions. The argument list is given in the kernel definitions at the top.
- You use cudaMemcpy() to copy h_input to the device and then use the same command to read it back on the host. Go to the web or gpt to ask how to do this.
- i did not know what intially what the shared memory case was. And then I looked in the kernel definition. There is a hint there for what they want.
- for the shared case and register case, you cannot copy directly from the host into shared or register memory. All you can do is copy d_input[i] into shared or register. Seems hard to understand how copying d_input[i] into shared and then test the shared value. Strange.
- Constant memory is perhaps the hardest case. Like global, you can copy from the host into global using cudaMemcpyToSymbol. The problem though is that there is no way to copy back from device constant memory back to the host.
- At the bottom of the instructions, there is a discussion of the memory. In the run.sh, we have max values of 128*64=8k elements. Now size of int is 4b and not 2 as mentioned. 8k*4b is 32kB for the max case which is less than the 64kB of constant memory. Regardless, I used trial and error to see that "-t 128 & -m 8" completes but "-t 128 -m 16" gives error during the copy.
- What you do for the large cases is that you only write to constant memory what can be written. Though it seems that -m 16,32,64 are being test, in reality, only "-m 8" is used for the large cases

Thanks for reading to the end. I hope it is useful

Like

Reply 1

+ Follow this post

More ...

1 Reply

ND Nick Decroos Learner

6 months ago

Hi Peter, do you have a copy of the starting *.cu file? This assignment takes me also much longer than expected.

Like

Reply 3

More ...

pb peter brooker Learner

6 months ago

sorry. I did not make a copy.

maybe I can help and in the process cement the concepts more.

can you successfully use cudaMemcpy() to move h_input into d_input and then, as a check, use cudaMemcpy() to copy back d_input into hd_input? The declaration of hd_input to receive d_input is very strange; int* hd_input = new int[numElements];

And I was serious. I have spent 5 days on this.

Like

More ...

G Gary Learner

6 months ago

I know that it has been 8 days and you may not need it. However, may help someone else. I believe that this is the starting one.

--- part 1 of 2

```
#include "memory_analysis.h"
```

```
__global__ void globalMemorySearch(int *input, int totalFound, int numElements, int numThreads)

{

    int threadId = blockDim.x * blockIdx.x + threadIdx.x;

    if (threadId < numThreads)

    {

        // Create a for loop that handles the fact that each thread needs to search for n values in input

        // Increment the input array at index i by 1, update in memory prior to testing if it is equal to constant_search_value

    }

}
```